

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1407

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:4 Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

I KAMALESH M (192421438) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

KAMALESH M

Course Faculty :Dr.N.DEEPA

Annexure A

DESIGN TASK

PROGRAM 1:

```
#include <stdio.h>

#include <string.h>
```

```
char input[50];
```

```
char stack[50];
```

```
int top = -1;
```

```
int ip = 0;
```

```
void push(char c) {

    stack[++top] = c;

}
```

```
char pop() {

    return stack[top--];

}
```

```
char peek() {

    return stack[top];

}
```

```
void printStack() {  
    for (int i = 0; i <= top; i++)  
        printf("%c", stack[i]);  
}
```

```
int main() {  
  
    printf("Grammar:\n");  
  
    printf("E->TA\n");  
    printf("A->+TA|e\n");  
    printf("T->FB\n");  
    printf("B->*FB|e\n");  
    printf("F->(E)|i\n\n");  
  
    printf("Enter input (use i for id): ");  
    scanf("%s", input);  
  
    strcat(input, "$");  
  
    push('$');  
    push('E');  
  
    printf("\nStack\t\tInput\t\tAction\n");  
  
    while (top != -1) {
```

```
printStack();
```

```
printf("\t\t%s\t\t", input + ip);
```

```
char x = peek();
```

```
char a = input[ip];
```

```
if (x == a && x == '$') {
```

```
    printf("Accepted\n");
```

```
    break;
```

```
}
```

```
else if (x == 'i' || x == '+' || x == '*' || x == '(' || x == ')' || x == '$') {
```

```
    if (x == a) {
```

```
        pop();
```

```
        ip++;
```

```
        printf("Match %c\n", a);
```

```
    } else {
```

```
        printf("Error\n");
```

```
        break;
```

```
    }
```

```
}
```

```
else if (x == 'E') {
```

```
    if (a == 'i' || a == '(') {
```

```
        pop();
```

```
        push('A');
```

```

    push('T');

    printf("E->TA\n");

} else {

    printf("Error\n");

    break;

}

}

else if (x == 'A') {

    if (a == '+') {

        pop();

        push('A');

        push('T');

        push('+');

        printf("A->+TA\n");

    } else if (a == ')' || a == '$') {

        pop();

        printf("A->e\n");

    } else {

        printf("Error\n");

        break;

    }

}

}

```

```

else if (x == 'T') {

    if (a == 'i' || a == '(') {

        pop();


```

```
    push('B');

    push('F');

    printf("T->FB\n");

} else {

    printf("Error\n");

    break;

}

}
```

```
else if (x == 'B') {

    if (a == '*') {

        pop();

        push('B');

        push('F');

        push('*');

        printf("B->*FB\n");

    } else if (a == '+' || a == ')' || a == '$') {

        pop();

        printf("B->e\n");

    } else {

        printf("Error\n");

        break;

    }

}
```

```
else if (x == 'F') {

    if (a == 'i') {
```

```
        pop();

        push('i');

        printf("F->i\n");
    } else if (a == '(') {

        pop();

        push('i');

        push('E');

        push('(');

        printf("F->(E)\n");

    } else {

        printf("Error\n");

        break;

    }

}

}

return 0;

}
```

Output:

```

Enter input (use i for id): i+i*i

Stack      Input      Action
$E         i+i*i$     E->TA
$AT        i+i*i$     T->FB
$ABF       i+i*i$     F->i
$ABi       i+i*i$     Match i
$AB        +i*i$     B->e
$A         +i*i$     A->+TA
$AT+       +i*i$     Match +
$AT        i*i$      T->FB
$ABF       i*i$      F->i
$ABi       i*i$      Match i
$AB        *i$       B->*FB
$ABF*      *i$       Match *
$ABF       i$        F->i
$ABi       i$        Match i
$AB        $         B->e
$A         $         A->e
$          $         Accepted

```

PROGRAM 2:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char input[50], stack[50];
```

```
int top = -1, i = 0;
```

```
void printState(char action[]) {
    printf("%-15s %-15s %s\n", stack, input + i, action);
}
```

```
int main() {
    printf("Grammar:\n");
    printf("E -> E+E\n");
    printf("E -> E*E\n");
    printf("E -> i\n");
}
```



```

printf("Enter input (use i for id): ");

scanf("%s", input);


strcat(input, "$");


printf("\n%-15s %-15s %s\n", "Stack", "Input", "Action");


while (1) {


    // Shift


    stack[++top] = input[i++];


    stack[top + 1] = '\0';


    printState("Shift");


    // Reduce i -> E


    if (top >= 0 && stack[top] == 'i') {


        stack[top] = 'E';


        printState("Reduce E->i");


    }


    // Reduce E * E -> E (Higher precedence)


    while (top >= 2 &&


        stack[top] == 'E' &&


        stack[top - 1] == '*' &&


        stack[top - 2] == 'E') {

```

```
top -= 2;

stack[top] = 'E';

stack[top + 1] = '\0';

printStats("Reduce E->E*E");

}
```

```
// Reduce E+E -> E
```

```
while (top >= 2 &&

    stack[top] == 'E' &&

    stack[top - 1] == '+' &&

    stack[top - 2] == 'E') {
```

```
// Delay reduction if next input is '*'
```

```
if (input[i] == '*')

    break;
```

```
top -= 2;

stack[top] = 'E';

stack[top + 1] = '\0';

printStats("Reduce E->E+E");

}
```

```
// Accept
```

```
if (strcmp(stack, "E") == 0 && input[i] == '$') {

    printf("String Accepted\n");

    break;
```

```
}
```

```
if (input[i] == '$' && strcmp(stack, "E") != 0) {
```

```
    while (top >= 2 &&
```

```
        stack[top] == 'E' &&
```

```
        stack[top - 1] == '+' &&
```

```
        stack[top - 2] == 'E') {
```

```
        top -= 2;
```

```
        stack[top] = 'E';
```

```
        stack[top + 1] = '\0';
```

```
        printState("Reduce E->E+E");
```

```
    }
```

```
if (strcmp(stack, "E") == 0) {
```

```
    printState("Accept");
```

```
    printf("\nString Accepted\n");
```

```
} else {
```

```
    printf("\nString Rejected\n");
```

```
}
```

```
break;
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

OUTPUT:

Enter input (use i for id): i+i*i

Stack	Input	Action
i	+i*i\$	Shift
E	+i*i\$	Reduce E->i
E+	i*i\$	Shift
E+i	*i\$	Shift
E+E	*i\$	Reduce E->i
E+E*	i\$	Shift
E+E*i	\$	Shift
E+E*E	\$	Reduce E->i
E+E	\$	Reduce E->E*E
E	\$	Reduce E->E+E
E	\$	Accept

String Accepted

PROGRAM 3:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char input[100];
```

```
char stack[100];
```

```
int top = -1, ip = 0;
```

```
void push(char x)
```

```
{
```

```
    stack[++top] = x;
```

```
}
```

```
void pop()
```

```
{
```

```
    top--;
```

```
}
```

```
void displayStack()

{

    for (int i = 0; i <= top; i++)

        printf("%c", stack[i]);

}


int main()

{

    printf("Grammar after removing Left Recursion:\n");

    printf("S -> TA\n");

    printf("A -> +TA | e\n");

    printf("T -> FB\n");

    printf("B -> *FB | e\n");

    printf("F -> (S) | i\n\n");


    printf("Enter input (use i for id): ");

    scanf("%s", input);


    strcat(input, "$");


    push('$');

    push('S');


    printf("\n%-15s %-15s %s\n", "Stack", "Input", "Action");


    while (top >= 0)

    {
```

```
displayStack();
```

```
printf("\t\t%s\t\t", input + ip);
```

```
char X = stack[top];
```

```
char a = input[ip];
```

```
// Accept
```

```
if (X == '$' && a == '$')
```

```
{
```

```
    printf("Accepted\n");
```

```
    printf("\nString Accepted\n");
```

```
    break;
```

```
}
```

```
// Match terminals
```

```
if (X == 'i' || X == '+' || X == '*' || X == '(' || X == ')' || X == '$')
```

```
{
```

```
    if (X == a)
```

```
    {
```

```
        pop();
```

```
        ip++;
```

```
        printf("Match %c\n", a);
```

```
    }
```

```
else
```

```
{
```

```
    printf("Error\n");
```

```
    break;
```

```
    }  
}  
  
// S -> TA  
  
else if (X == 'S')  
{  
    if (a == 'i' || a == '(')  
    {  
        pop();  
        push('A');  
        push('T');  
        printf("S->TA\n");  
    }  
    else  
    {  
        printf("Error\n");  
        break;  
    }  
}
```

```
// A -> +TA | e  
  
else if (X == 'A')  
{  
    if (a == '+')  
    {  
        pop();  
        push('A');
```

```
    push('T');

    push('+');

    printf("A->+TA\n");
}

else if (a == ')' || a == '$')

{

    pop();

    printf("A->e\n");

}

else

{

    printf("Error\n");

    break;

}

}
```

// T -> FB

```
else if (X == 'T')

{

    if (a == 'i' || a == '(')

    {

        pop();

        push('B');

        push('F');

        printf("T->FB\n");

    }

    else
```



```
{  
  
    printf("Error\n");  
  
    break;  
  
}  
  
}
```

```
// B -> *FB | e
```

```
else if (X == 'B')
```

```
{  
  
    if (a == '*')  
  
    {  
  
        pop();  
  
        push('B');  
  
        push('F');  
  
        push('*');  
  
        printf("B->*FB\n");  
  
    }
```

```
else if (a == '+' || a == ')' || a == '$')
```

```
{  
  
    pop();  
  
    printf("B->e\n");  
  
}
```

```
else
```

```
{  
  
    printf("Error\n");  
  
    break;  
  
}
```

```
}
```

```
// F -> i | (S)
```

```
else if (X == 'F')
```

```
{
```

```
    if (a == 'i')
```

```
    {
```

```
        pop();
```

```
        push('i');
```

```
        printf("F->i\n");
```

```
    }
```

```
else if (a == '(')
```

```
{
```

```
    pop();
```

```
    push(')');
```

```
    push('S');
```

```
    push('(');
```

```
    printf("F->(S)\n");
```

```
}
```

```
else
```

```
{
```

```
    printf("Error\n");
```

```
    break;
```

```
}
```

```
}
```

```
}
```

```

return 0;

}

```

OUTPUT:

```

Enter input (use i for id): i+i*i

Stack      Input      Action
$S          i+i*i$      S->TA
$AT         i+i*i$      T->FB
$ABF        i+i*i$      F->i
$ABi        i+i*i$      Match i
$AB         +i*i$       B->e
$A          +i*i$       A->+TA
$AT+        +i*i$       Match +
$AT         i*i$        T->FB
$ABF        i*i$        F->i
$ABi        i*i$        Match i
$AB         *i$         B->*FB
$ABF*       *i$         Match *
$ABF        i$          F->i
$ABi        i$          Match i
$AB         $           B->e
$A          $           A->e
$           $           Accepted

String Accepted

```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation

Program4:

Designing an unambiguous grammar with custom operators (1)

Task: Create CFG for %, ^, +, - with user-defined precedence.

1) Define Precedence & Associative Hierarchically

OPERATOR
^ (R, RIGHTEST)
% (L)
+ (L)
- (L) LOWEST

2) Eliminate ambiguity (Left Recursion)

Removing Left Recursion

$E \rightarrow TE'$

$E' \rightarrow +TE' | \epsilon$

Ambiguous: $E \rightarrow E+E | E \times E | id$

Resolve: use different non-terminal

Expr $\rightarrow$ Expr + Term | Expr - Term | Term

Term $\rightarrow$ Term % Factor | Factor

Factor $\rightarrow$ Atom ^ Factor | Atom

Atom $\rightarrow id | (Expr)$

* LEFT ASSOC: $E \rightarrow E+T$, $E' id$: $F \rightarrow A^N F$

* RIGHT ASSOC: $E \rightarrow T+E'$

* Validation & Demo

FIRST/FOLLOW

FIRST FOLLOW

Input: $id + id \% id \wedge id$

Non-terminal	id	+	%	^
Expr	1	2	4/2	1
Term	1	3	3/3	2
Factor	0	1	4	5

STACK	INPUT	ACTION
SHIFT	id	shift id
REDUCE	Atom $\rightarrow id$	PARSE TREE
PARSE TREE	Atom $\rightarrow id$	grow!